

CHAPITRE VII

Algorithme glouton

Algorithme glouton

- ❖ Choix d'activité
- ❖ Les pièces de monnaie
- ❖ Problème du sac à dos
- ❖ Codage de Huffman

Un ***algorithme glouton*** fait toujours le choix qui lui semble le meilleur.

Les algorithmes gloutons n'aboutissent pas toujours à des solutions optimales, mais ils y arrivent dans de nombreux cas.

Algorithme glouton

❖ Choix d'activité:

Supposez que l'on ait un ensemble $S = \{a_1, a_2, \dots, a_n\}$ de n **activités** proposées qui veulent utiliser une ressource, par exemple une salle de conférences qui ne peut servir qu'à une seule activité à la fois. Chaque activité a_i a une **heure de début** s_i et une **heure de fin** f_i , avec $0 \leq s_i < f_i < \infty$.

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

Algorithme glouton

❖ Les pièces de monnaie: rendu de monnaie.

Etant donné un système de monnaie (pièces), comment rendre une somme donnée de façon optimale, c'est-à-dire avec le nombre minimal de pièces?

Exemple:

Avec l'ensemble de monnaies en euro $\{1, 2, 5, 10, 20, 50, 100, 200\}$, il semble nous donner toujours une solution optimale. Par exemple pour rendre 10 il retourne $\{10\}$, et pour 24 il retourne $\{20, 2, 2\}$.

Avec l'ensemble de monnaies du franc CFA $\{5, 10, 25, 50, 100, 200, 250, 500\}$ Par exemple pour rendre 400 il retourne $\{250, 100, 50\}$.

Comment rendre 10 avec l'ensemble de monnaies $\{1, 4, 6, 7\}$ en utilisant l'algorithme glouton? Est-il le meilleur choix

Algorithme glouton

❖ Problème du sac à dos.

Durant un cambriolage un voleur possède un sac dont la capacité est limitée. Il se trouve face à n objets qu'il peut dérober. Chacun de ces objets est caractérisé par sa valeur v_i et son poids w_i . Il veut que son butin ait la plus grande valeur possible mais il ne peut pas transporter plus de W kilos dans son sac à dos.



Quels objets devra-t-il prendre pour maximiser son butin?

❖ La variante **TOR**

❖ La variante fractionnaire(Frac).

Algorithme glouton

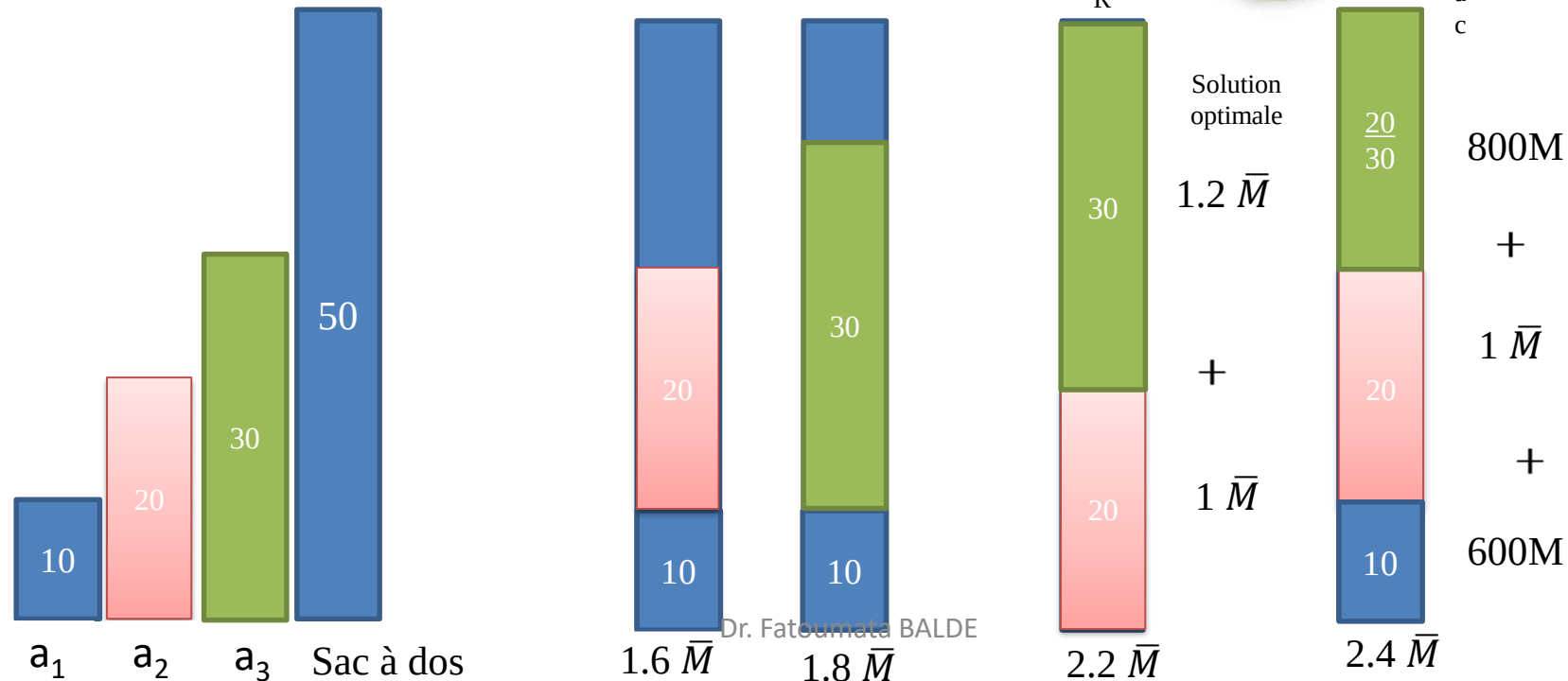
Glouton peut résoudre Frac en $O(\log n)$ mais pas TOR.

Exemple: Il existe 3 types d'articles. (Poids du sac 50kg)

article 1 pèse 10 kilos et vaut 600.000Fcfa soit 60M/kg

article 2 pèse 20 kilos et vaut 1.000.000Fcfa soit 50M/kg

article 3 pèse 30 kilos et vaut 1.200.000Fcfa soit 40M/kg



Algorithme glouton

❖ Codage de Huffman: Compression de données

Technique très efficace de compression de données.

On utilise un tableau contenant les fréquences d'apparition de chaque caractère pour établir une manière optimale de représenter chaque caractère par une chaîne binaire.

❖ Des économies de 20% à 90% sont courantes.

Exemple: Soit un fichier de 100 000 caractères qu'on souhaite conserver de manière compacte.

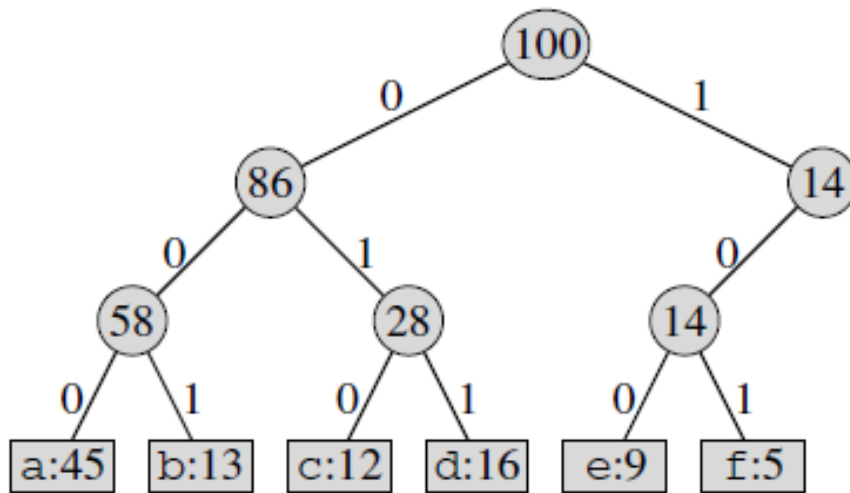
Le tableau suivant montre les fréquences d'apparition des caractères.

	a	b	c	d	e	f
Fréquence (en milliers)	45	13	12	16	9	5
Mot de code de longueur fixe	000	001	010	011	100	101
Mot de code de longueur variable	0	101	100	111	1101	1100

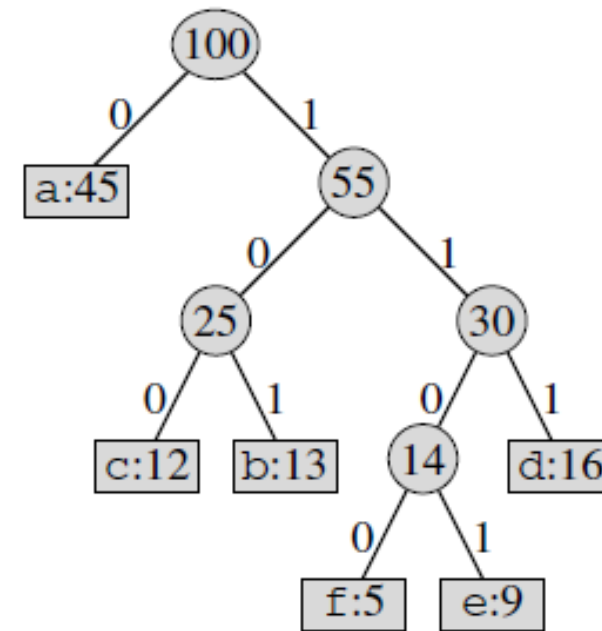
Algorithme glouton

❖ Codage de Huffman: Compression de données

On interprète le mot de code binaire associé à un caractère comme étant le chemin allant de la racine à ce caractère, où 0 signifie « bifurquer vers l'enfant gauche » et 1 signifie « bifurquer vers l'enfant droite ».



(a)



(b)

CHAPITRE VIII

Programmation dynamique

Programmation dynamique

C'est une méthode utile pour obtenir une **solution exacte** à un **problème algorithmique**, là où une solution "classique" se trouve être **trop complexe**.

La programmation dynamique, comme la méthode diviser pour régner résout les problèmes en combinant les solutions de sous-problèmes.

Exemples:

- ❖ Tas de Fibonacci
- ❖ Découpe de Barres
- ❖ PLSC

Programmation dynamique

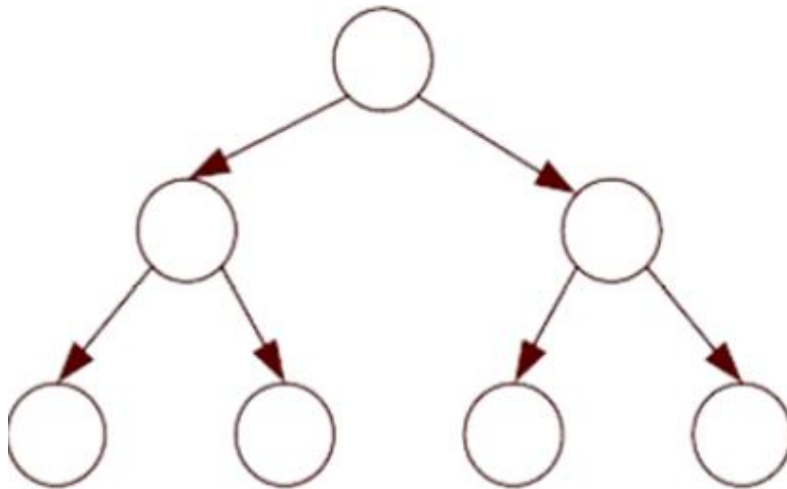
Souvent utilisée lorsqu'une solution récursive se révèle inefficace.

1. Solution récursive: Haut en bas.
2. Programmation dynamique: Bas en haut.

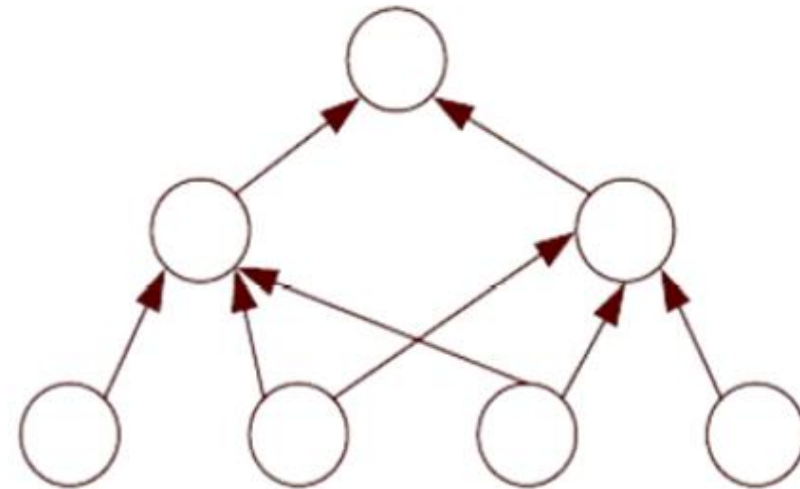
On évite de calculer plusieurs fois la même chose.



Richard Bellman



diviser et régner



programmation dynamique

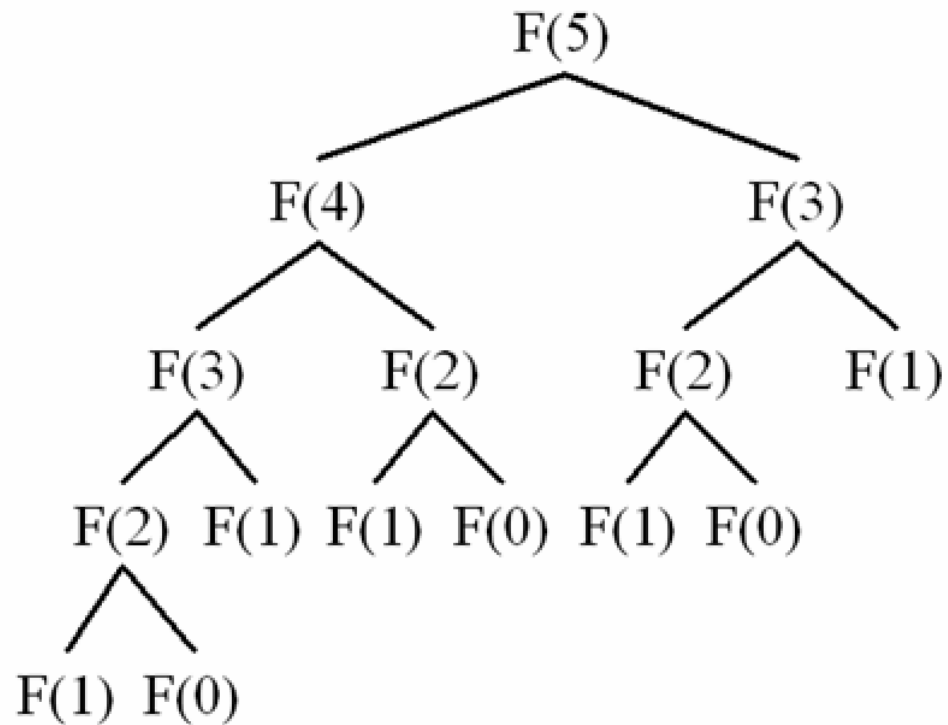
Programmation dynamique

Exemple: Tas de Fibonacci.

$F(0) = 1; F(1) = 1;$

$F(n) = F(n-1) + F(n-2)$

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89



Leonardo da Pisa, dit Fibonacci

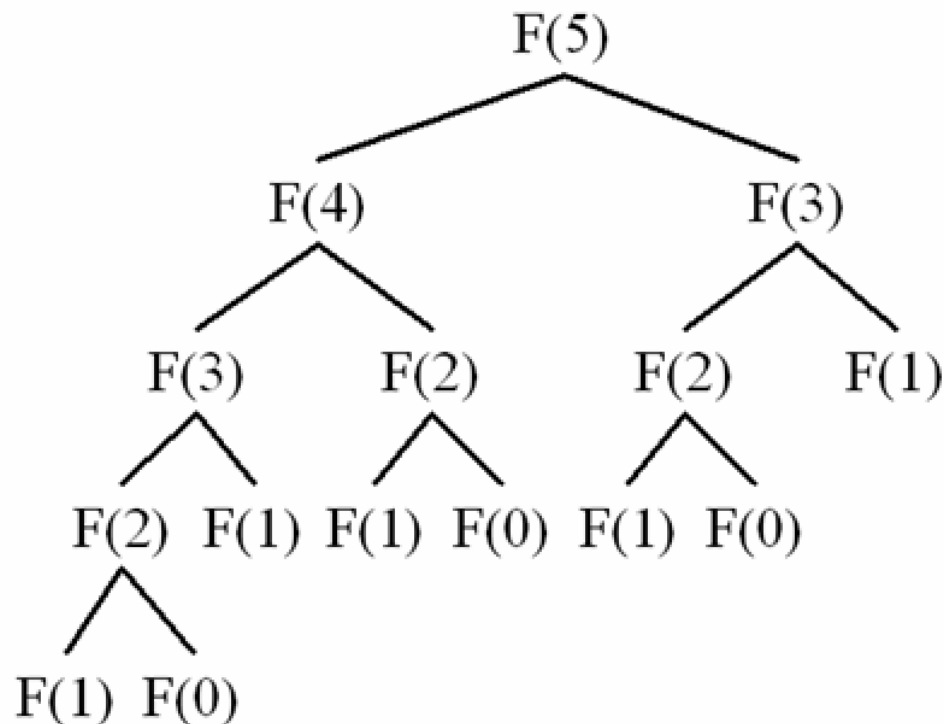
Programmation dynamique

Idée :

- ❖ Sauvegarder les résultats obtenus
- ❖ Vérifier si un résultat a déjà été obtenu avant de la calculer

Algorithme naif de la suite de Fibonacci

```
fibonacci(n) {  
    if (n <= 1) {  
        return 1  
    }  
    else {  
        return (fibonacci(n-1) + fibonacci(n-2))  
    }  
}
```



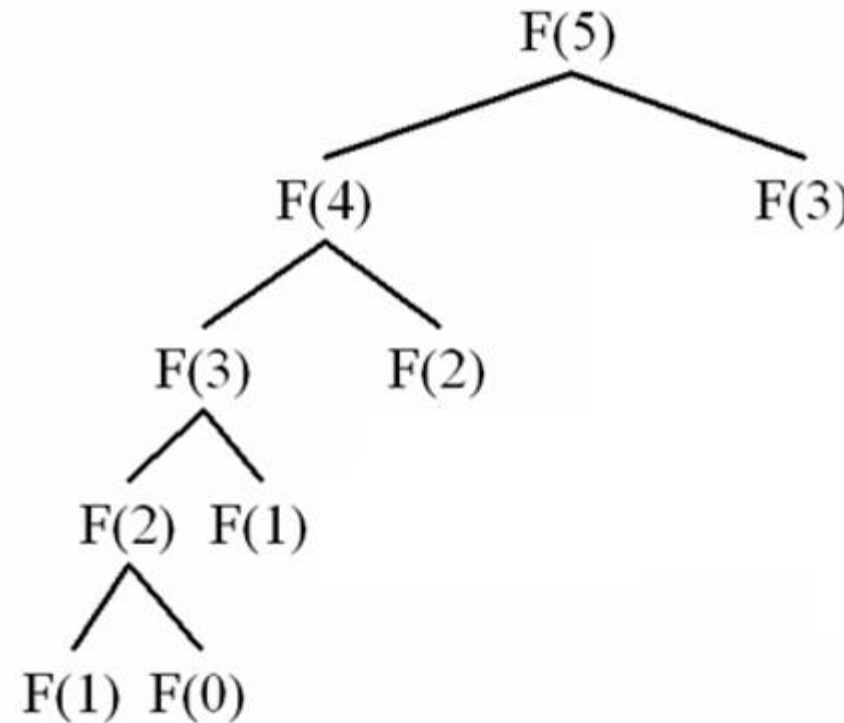
$O(2^n)$

Programmation dynamique

Améliorer la complexité temporelle

- ❖ Utiliser la forme récursive et sauvegarder le résultat dans un tableau (**mémoïsation ou Top Down**)

```
function fibo(n) {  
    if (table[n] existe)  
    { return table[n] }  
    if (n <= 2)  
    { return 1 }  
    else {  
        res = fibo(n-1) + fibo(n-2)  
        table[n] = res  
        return res } } }
```



Programmation dynamique

Algorithme non récursif ou Bottom Up. Il s'agit d'écrire cet algorithme de manière ascendante, dans une forme typique de la programmation dynamique :

```
fibo(n) {  
    tab[0]=1  
    tab[1]=1  
    for(i → 2 à n )  
        tab[i]=tab[i-1]+tab[i-2]  
    return tab[n]  
}
```

Programmation dynamique

❖ Découpe de Barres:

❖ Problème à résoudre:

❖ Barre de longueur fixe à couper pour revente

❖ **Objectif:** maximiser le gain.

Longueur	0	1	2	3	4	5	6	7	8
Prix	0	2	3	8	10	13	15	16	21

❖ Une solution optimale : bout de 3 + bout de 1 $\rightarrow$ 10.

❖ Autre solution optimale : bout de 4 $\rightarrow$ prix : 10

Programmation dynamique

❖ **Découpe de Barres:**

❖ Solution:

- ❖ Trouver toutes les découpes possibles.
- ❖ Calculer pour chacune le gain.
- ❖ Retourner le plus grand.

Comment calculer toutes les découpes possibles?

Programmation dynamique

❖ Découpe de Barres: $O(2^{\text{taille}-1})$

couperBarre (taille) :

si taille $\leq$ 0 :

retourner 0

gain= 0 *// meilleur prix.*

tc= 1 *// taille coupée.*

tant que tc $\leq$ taille

gain= max (gain , prix [tc] + couperBarre (taille – tc))

tc=tc+1

retourner gain

Programmation dynamique

❖ Memoïsation: (haut-bas) polynomial.

prixMemo = [-1,-1,-1,-1,-1,-1,-1,-1,-1]

couperBarre (taille) :

 si taille <= 0 :

 retourner 0

 si prixMemo[taille]>=0:

 retourner prixMemo[taille]

 gain= 0

// meilleur prix.

 tc= 1

// taille coupée.

 tant que tc <= taille

 gain= max (gain , prix [tc] + couperBarre (taille – tc))

 tc=tc+1

prixMemo[taille]=gain

retourner gain

Programmation dynamique

❖ Version naïf: (bas-haut) polynomial

PrixMémo = [0,0,0,0,0,0,0,0,0]

couperBarre (taille) :

i=0

 tant que *i* <= taille

 gain= 0

j=1

 tant que *j* <= *i*

 gain=max (gain , prix [*j*] +prixMemo[*i-j*])

j=*j*+1

 prixMemo[*i*]=gain

i=*i*+1

retourner prixMemo[taille]

Programmation dynamique

❖ PLSC(Plus Longue Sous-séquence Commune):

❖ Problème:

Le problème de la plus longue sous-séquence commune consiste, étant données deux séquences $X = (x_1, x_2, \dots, x_m)$ et $Y = (y_1, y_2, \dots, y_m)$, à déterminer une sous-séquence¹ Z commune à X et Y qui soit de longueur maximale.

Par exemple, si $X=(A, B, C, B, D, A, B)$ et $Y=(B, D, C, A, B, A)$ sont les séquences considérées, $Z=(B, C, B, A)$ est une solution, mais elle n'est pas forcément unique : (B, D, A, B) ou (B, C, A, B) sont d'autres PLSC...

Programmation dynamique

❖ PLSC(Plus Longue Sous-séquence Commune):

❖ Solution:

Si on note $c[i, j]$ la longueur de la plus longue sous-séquence commune à X_i et Y_j , on a :

$$c[i,j]=\begin{cases} 0 & \text{si } i = 0 \text{ ou } j = 0 \\ 1 + c[i - 1, j - 1] & i, j > 0 \text{ et } x_i = y_j \\ \max(c[i - 1, j], c[i, j - 1]) & i, j > 0 \text{ et } x_i \neq y_j \end{cases}$$

Programmation dynamique

❖ PLSC(Plus Longue Sous-séquence Commune):

❖ Algorithme PLSC

```
 $m \leftarrow \text{Longueur}(X)$   
 $n \leftarrow \text{Longueur}(Y)$   
pour  $i = 1$  to  $m$  faire  
     $c[i, 0] \leftarrow 0$   
Fin pour  
pour  $j = 0$  to  $n$  faire  
     $c[0, j] \leftarrow 0$   
Fin pour
```

```
pour  $i = 1$  to  $m$  faire  
    pour  $j = 1$  to  $n$  faire  
        si  $x_i = y_j$  alors  
             $c[i, j] \leftarrow c[i - 1, j - 1] + 1$   
             $b[i, j] \leftarrow "$  " "  
        sinon si  $c[i - 1, j] > c[i, j - 1]$   
            alors  
                 $c[i, j] \leftarrow c[i - 1, j]$   
                 $b[i, j] \leftarrow "$   $\uparrow$  "  
            sinon  
                 $c[i, j] \leftarrow c[i, j - 1]$   
                 $b[i, j] \leftarrow "$   $\leftarrow$  "  
        Fin si  
    Fin pour  
Fin pour  
retourner  $c$  et  $b$   
Complexité:  $O(nm)$ 
```

CHAPITRE IX

Algorithmes pour les graphes

Algorithmes élémentaires pour les graphes

❖ Définition:

un **graphe** est défini par un couple $G = (S, A)$ tel que:

- ❖ S est un ensemble fini de sommets (aussi appelés nœuds).
- ❖ $A \subseteq S \times S$ est un ensemble de couples de sommets définissant une relation binaire sur S .

Un graphe peut être orienté ou non orienté.

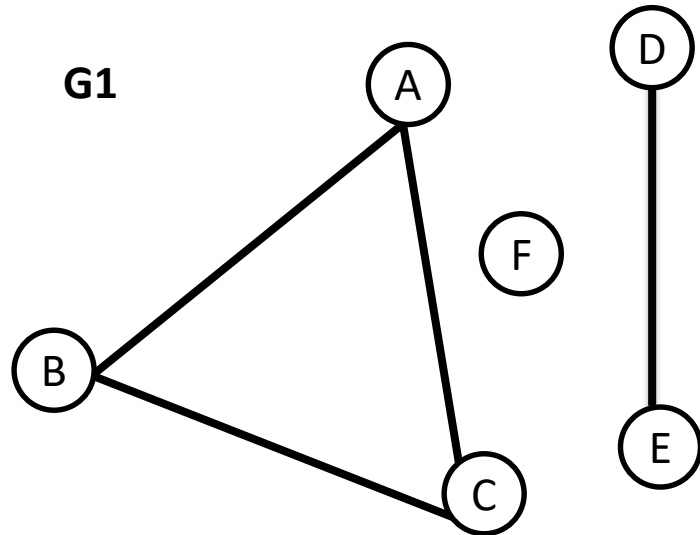
Graphe non orienté: $G = (S, A)$ est non orienté si $\forall (s_i, s_j) \in S \times S$,
 $(s_i, s_j) \in A \Leftrightarrow (s_j, s_i) \in A$

Graphe orienté: $G = (S, A)$ est orienté si $\exists (s_i, s_j) \in S \times S$, $(s_i, s_j) \in A$
et $(s_j, s_i) \notin A$

Algorithmes élémentaires pour les graphes

❖ Définition:

Exemple de graphe non orienté



$S = \{A, B, C, D, E, F\}$

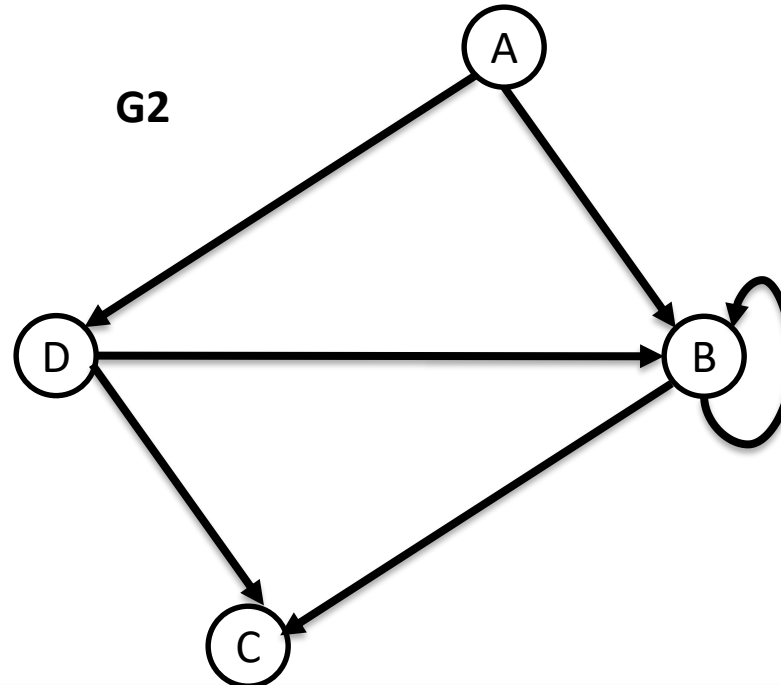
$A = \{(A, B), (B, A), (A, C), (C, A), (B, C), (C, B), (D, E), (E, D)\}$

Les éléments de A sont appelés arêtes.

s_i adjacent à s_j si $(s_i, s_j) \in A$.

degré d'un sommet = nombre de sommets adjacents

Exemple de graphe orienté



$S = \{A, B, C, D\}$

$A = \{(A, B), (A, D), (D, C), (B, C), (D, B), (B, B)\}$

Les éléments de A sont appelés arcs.

s_j successeur de s_i si $(s_i, s_j) \in A$.

s_j prédécesseur de s_i si $(s_j, s_i) \in A$.

Algorithmes élémentaires pour les graphes

❖ Définition: Cheminement et connexité.

Chaîne: Séquence de sommets $\langle s_0, s_1, \dots, s_k \rangle$ (notée $s_0 \sim s_k$) telle que $\forall i \in [1, k], (s_{i-1}, s_i) \in A$.

Connexité: Un graphe non orienté est **connexe** s'il y a une chaîne entre n'importe quelle paire de sommets distincts du graphe. 771213816

Longueur d'une chaîne / d'un chemin: nombre d'arêtes /d'arcs dans la chaîne/chemin.

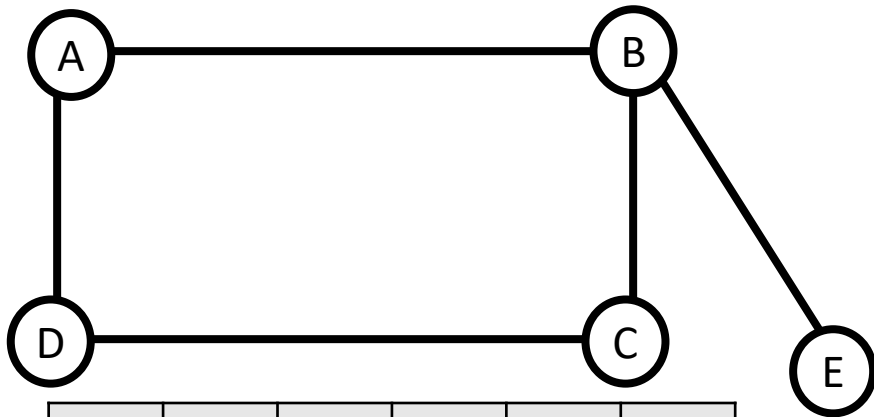
Cycle/Circuit = Chaîne / Chemin commençant et terminant par un même sommet.

Algorithmes élémentaires pour les graphes

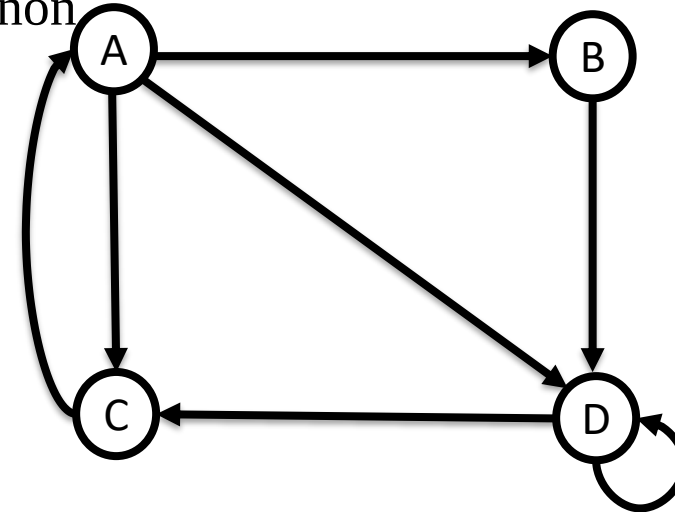
❖ Définition: Matrice d'adjacence.

La matrice d'adjacence d'un graphe $G(S, A)$ est une matrice M telle que

$M[s_i][s_j] = 1$ si $(s_i, s_j) \in A$, et $M[s_i][s_j] = 0$ sinon



M	A	B	C	D	E
A	0	1	0	1	0
B	1	0	1	0	1
C	0	1	0	1	0
D	1	0	1	0	0
E	0	1	0	0	0

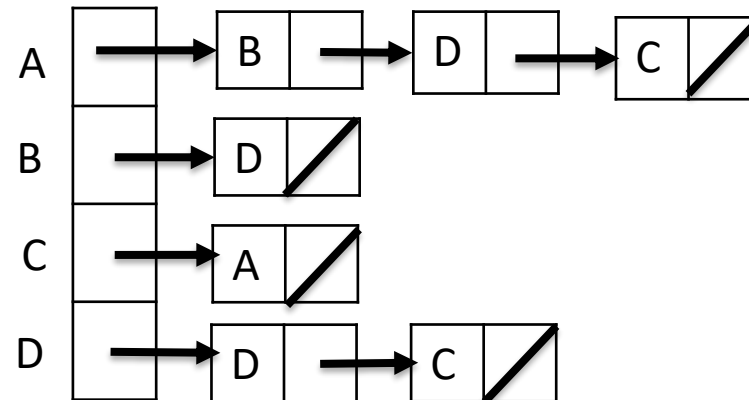
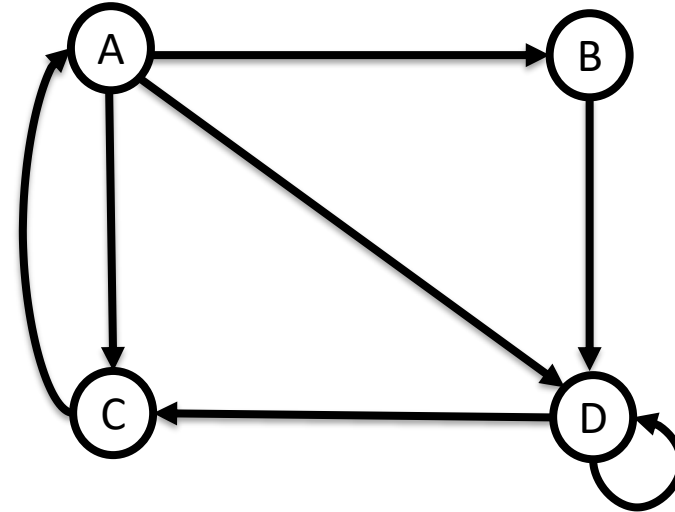
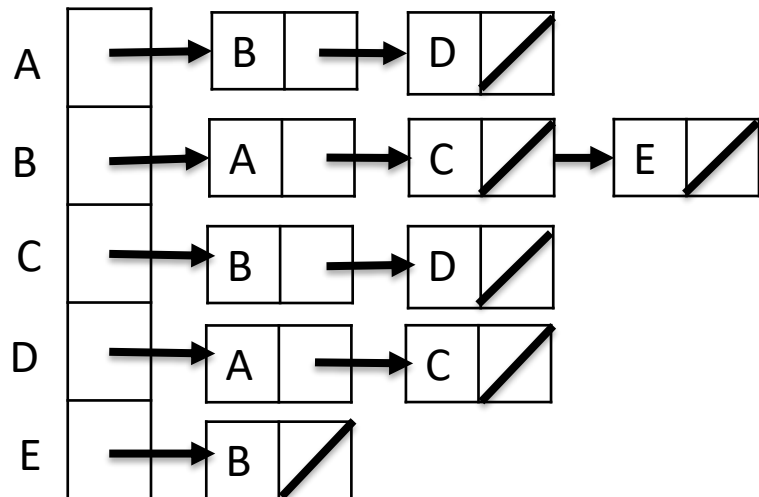
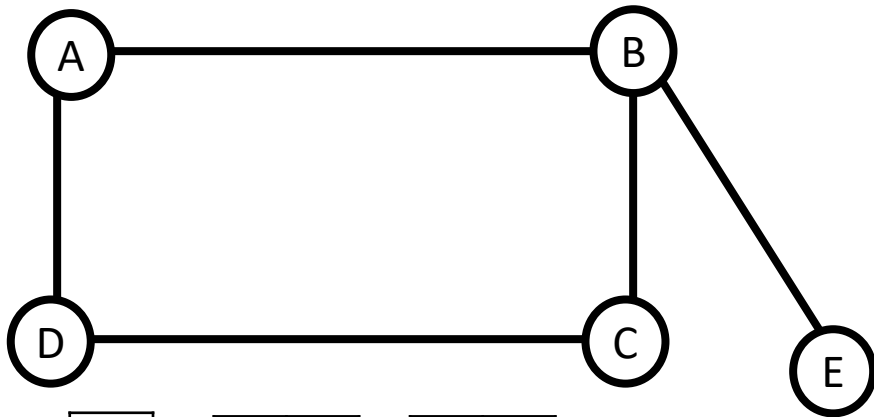


M	A	B	C	D
A	0	1	1	1
B	0	0	0	1
C	1	0	0	0
D	0	0	1	1

Algorithmes élémentaires pour les graphes

❖ Définition: Liste d'adjacence.

La liste d'adjacence d'un graphe $G(S, A)$ est Tableau succ tel que $\text{succ}[s_i] = \text{liste des successeurs de } s_i$



Algorithmes élémentaires pour les graphes

❖ **Parcours de graphe:**

Parcourir un graphe c'est visiter tous les sommets du graphe à partir d'un sommet de départ.

Marquage des sommets:

Un sommet est colorié en **blanc** s'il n'est pas encore visité.

Un sommet est colorié en **gris** s'il est en cours d'exploitation.

Un sommet est colorié en **noire** s'il est déjà exploité.

On utilise une file pour le parcours en largeur et une pile pour le parcours en profondeur.

Algorithmes élémentaires pour les graphes

❖ Algorithme de Parcours en largeur(BFS):

Le parcours en largeur consiste à parcourir d'abord tous les voisins d'un sommet donné, puis on parcourt les voisins des voisins, etc..

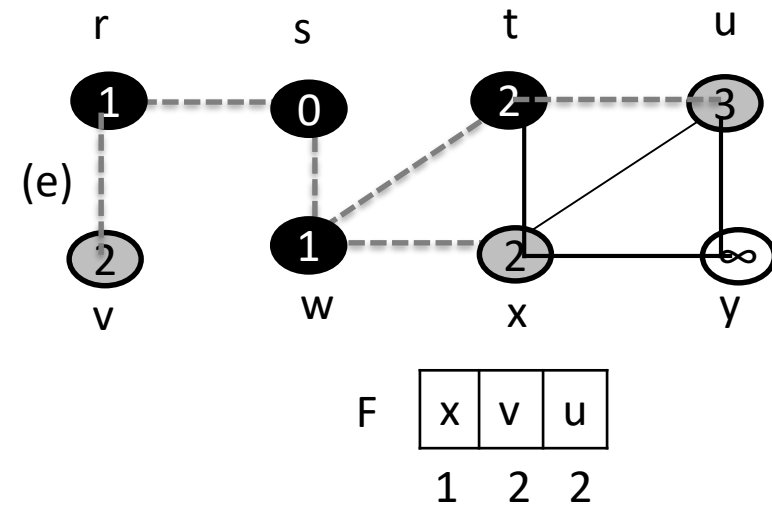
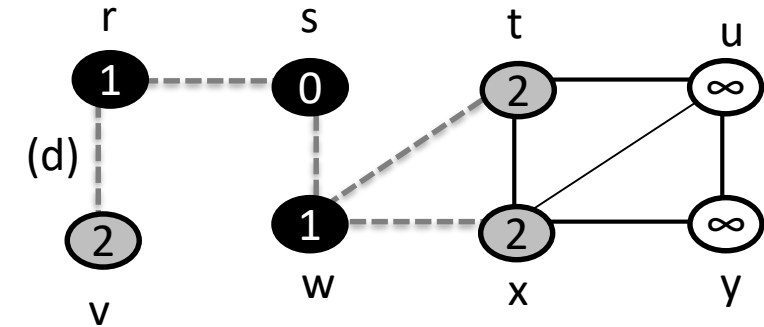
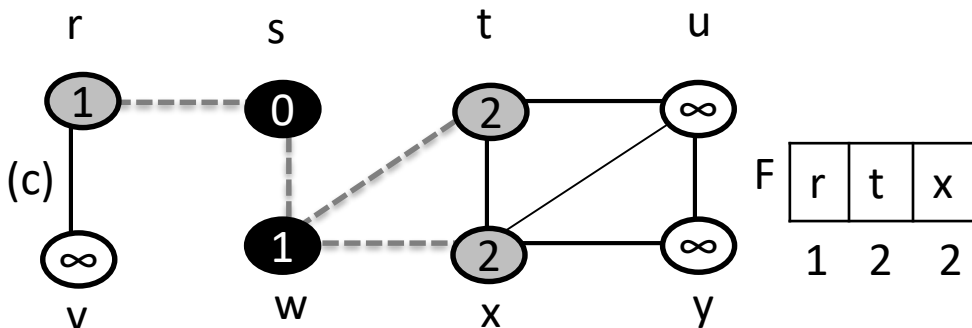
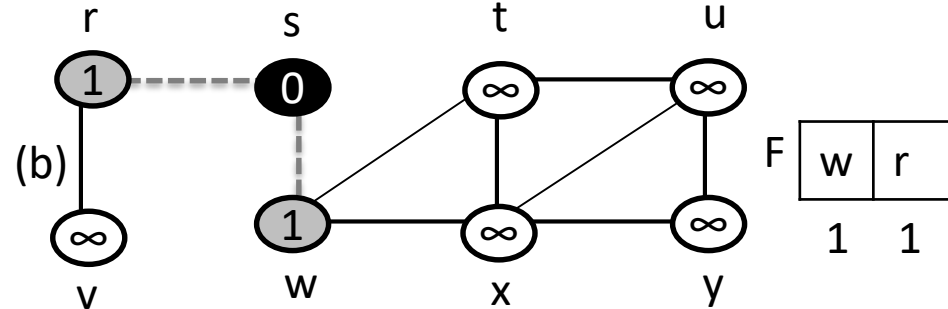
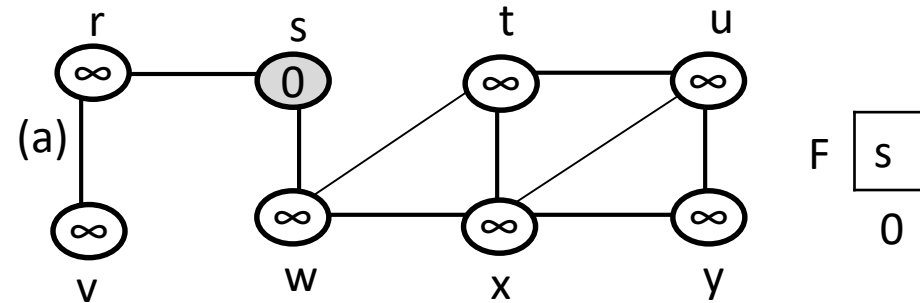
Pour un graphe de S sommets et A arêtes,

- ❖ Initialiser les informations : $O(S)$
- ❖ Chaque sommet est placé une seule fois dans la queue: $O(S)$
- ❖ Chaque sommet est supprimé une seule fois de la file: $O(S)$
- ❖ Pour chaque sommet supprimé de la file, on examine tous les voisins: $O(A)$
- ❖ Temps d'exécution pour toutes ces étapes est **$O(S+A)$**

```
PL(G,s)
  Pour chaque sommet  $u \in G.S - \{s\}$ 
     $u.couleur = BLANC$ 
     $u.d = \infty$ 
     $u.\pi = NIL$ 
   $s.couleur = GRIS$ 
   $s.d = 0$ 
   $s.\pi = NIL$ 
   $F = \emptyset$ 
  Enfiler( $F,s$ )
  Tant que  $F \neq \emptyset$ 
     $U = Defiler(F)$ 
    Pour chaque  $v \in G.Adj[u]$ 
      si  $v.couleur == BLANC$ 
         $v.couleur = GRIS$ 
         $v.d = u.d + 1$ 
         $v.\pi = u$ 
        Enfiler( $F,v$ )
     $u.couleur = NOIR$ 
```

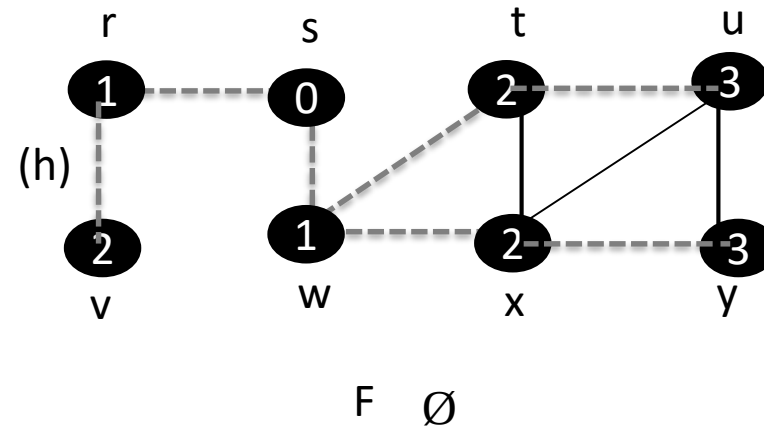
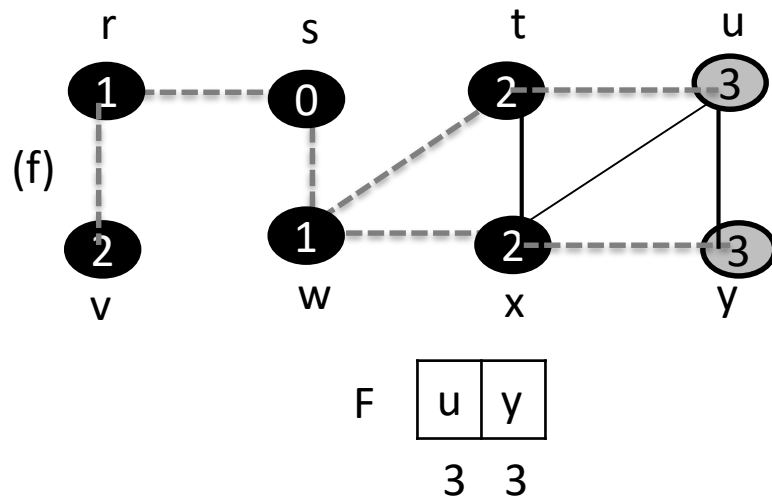
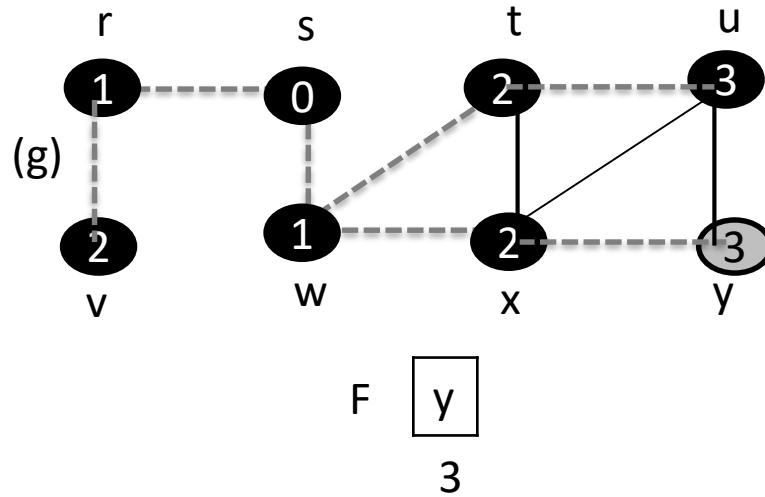
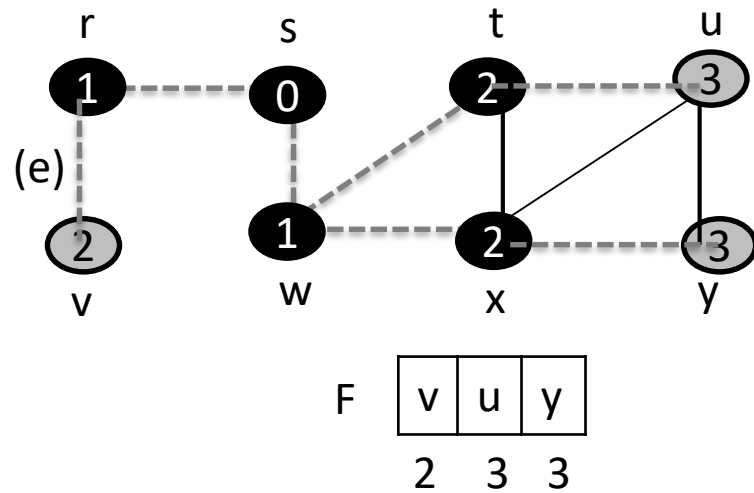
Algorithmes élémentaires pour les graphes

❖ Illustration: Parcours en largeur



Algorithmes élémentaires pour les graphes

❖ Illustration: Parcours en largeur



Algorithmes élémentaires pour les graphes

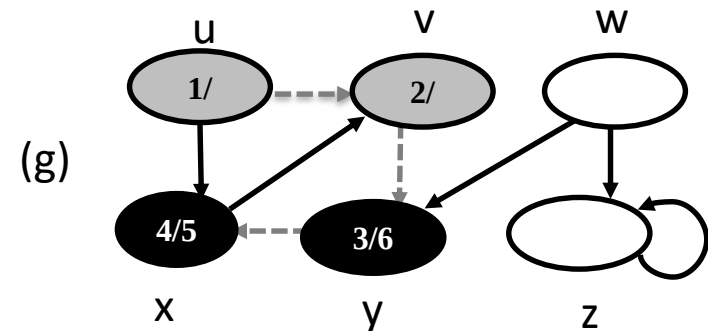
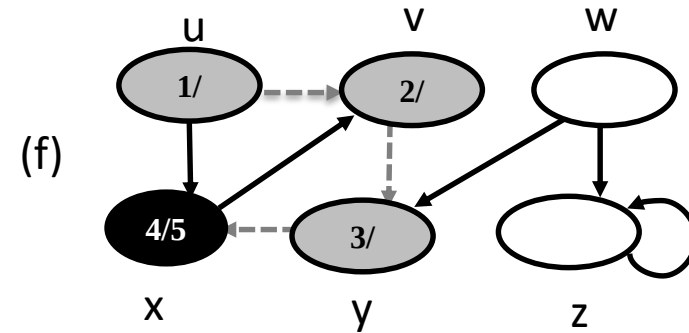
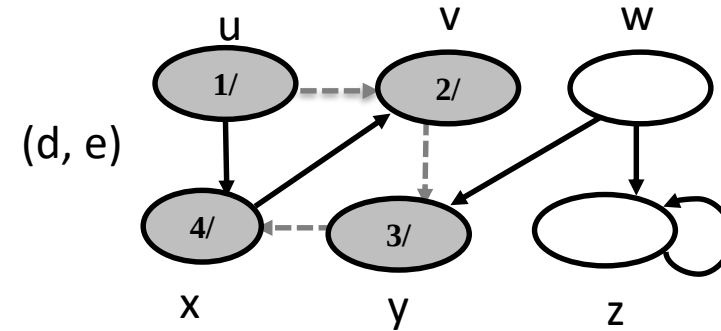
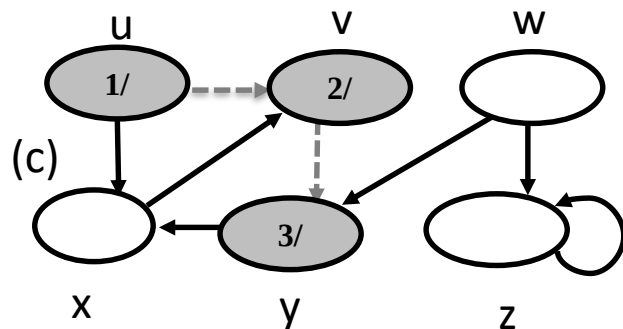
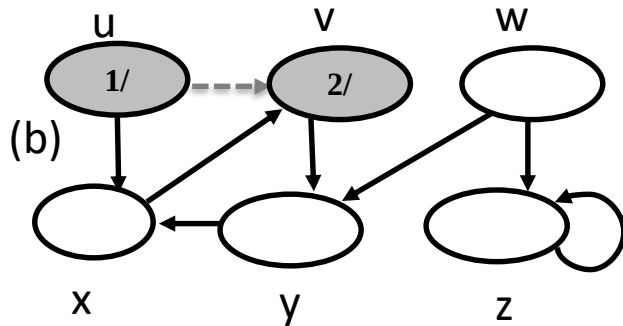
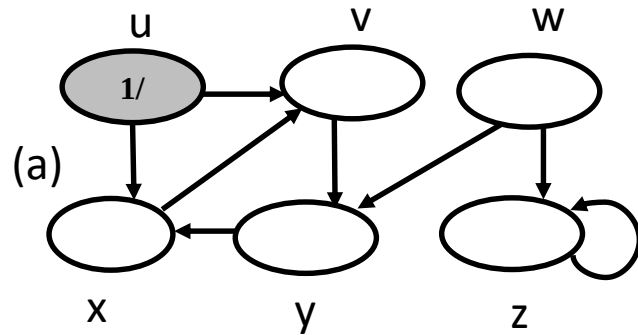
- ❖ **Algorithme de Parcours en Profondeur(BFS):** Le parcours en profondeur Consiste à descendre plus « profondément » dans le graphe chaque fois que c'est possible, les arcs sont explorés à partir du sommet v découvert le plus récemment et dont on n'a pas encore exploré tous les arcs qui en partent.

```
PP(G)
Pour chaque sommet  $u \in G.S$ 
     $u.couleur = BLANC$ 
     $u.\pi = NIL$ 
date = 0
Pour chaque sommet  $u \in G.S$ 
    Si  $u.couleur = BLANC$ 
        Visiter_PP(G,u)
```

```
Visiter_PP(G,u)
    date = date+1 //sommet blanc trouve
     $u.d = date$ 
     $u.couleur = GRIS$ 
    Pour chaque  $v \in G.Adj[u]$  //explore arc(u,v)
        si  $v.couleur == BLANC$ 
             $v.\pi = u$ 
            Visiter_PP(G,v)
     $u.Couleur = NOIR$ 
    date = date+1
     $u.f = date$ 
```

Algorithmes élémentaires pour les graphes

❖ Illustration: Parcours en largeur



Algorithmes élémentaires pour les graphes

❖ Illustration: Parcours en largeur

